Question 1.)

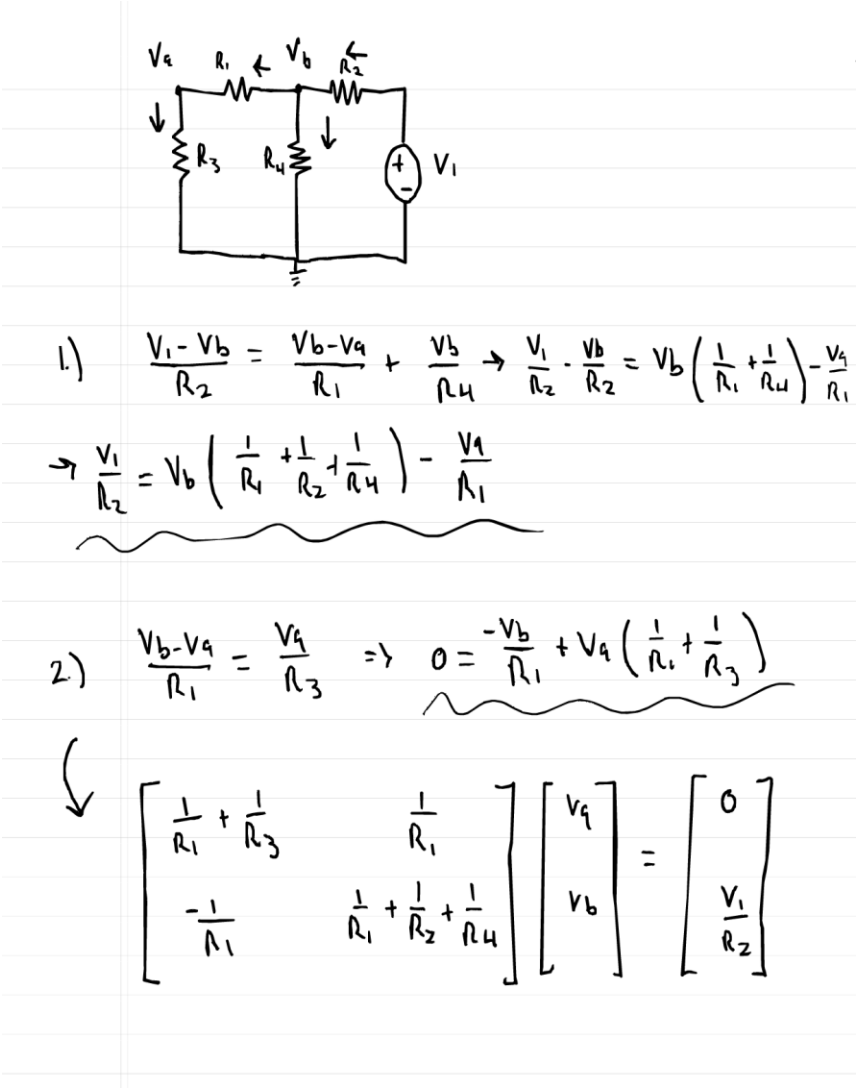


Fig 1.) – Worked out voltage equations

From code, both linalg solve and my custom gaussian elimination method, Va = 7.5V and Vb = 8.333V

Code:

```
import numpy as np


v1 = 10

R1 = 1000

R2 = 1000
```

```
R3 = 9000
R4 = 10000

A=np.array([
   [1/R1 + 1/R3, -1/R1],
        [-1/R1, 1/R1 + 1/R2 + 1/R4]])

b = np.array([0, v1 / R2])

v = np.linalg.solve(A, b)
va, vb = v
print(f"Node voltages from numpy: va = {va:.4f}V, vb = {vb:.4f}V")

def gaussian_elim(A, b):
   A= A.astype(float)
   b = b.astype(float)
   n = len(b)

   for k in range(n):
      maxindex = abs(A[k:, k]).argmax()+k #Looks at all entries in column k, takes abs value, returns index of largest
      if A[maxindex, k] == 0:
         raise ValueError("Matrix is singular") #If pivot is 0, raise an error
      if maxindex !=k:
         A[[k, maxindex]] = A[[maxindex, k]]
         b[[k, maxindex]] = b[[maxindex, k]] #Swaps rows

      for row in range(k+1, n):
         mult = A[row, k] / A[k, k]
```

```
            A[row, k:] -= mult* A[k,k:]
            b[row]-= mult * b[k] #Eliminates to create upper diag.

    x = np.zeros(n)
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - np.dot(A[i, i+1:], x[i+1:])) / A[i, i]
    return x #Back substitution for solutions

v_gauss = gaussian_elim(A.copy(), b.copy())
print(f"Homemade node voltages: va = {v_gauss[0]:.4f}V, vb = {v_gauss[1]:.4f}V")

if(va == v_gauss[0] and vb==v_gauss[1]):
    print("Voltages match!")
else:
    print("Voltages are off")
```

Q2.)

Heatmaps for m=10 and m=25:

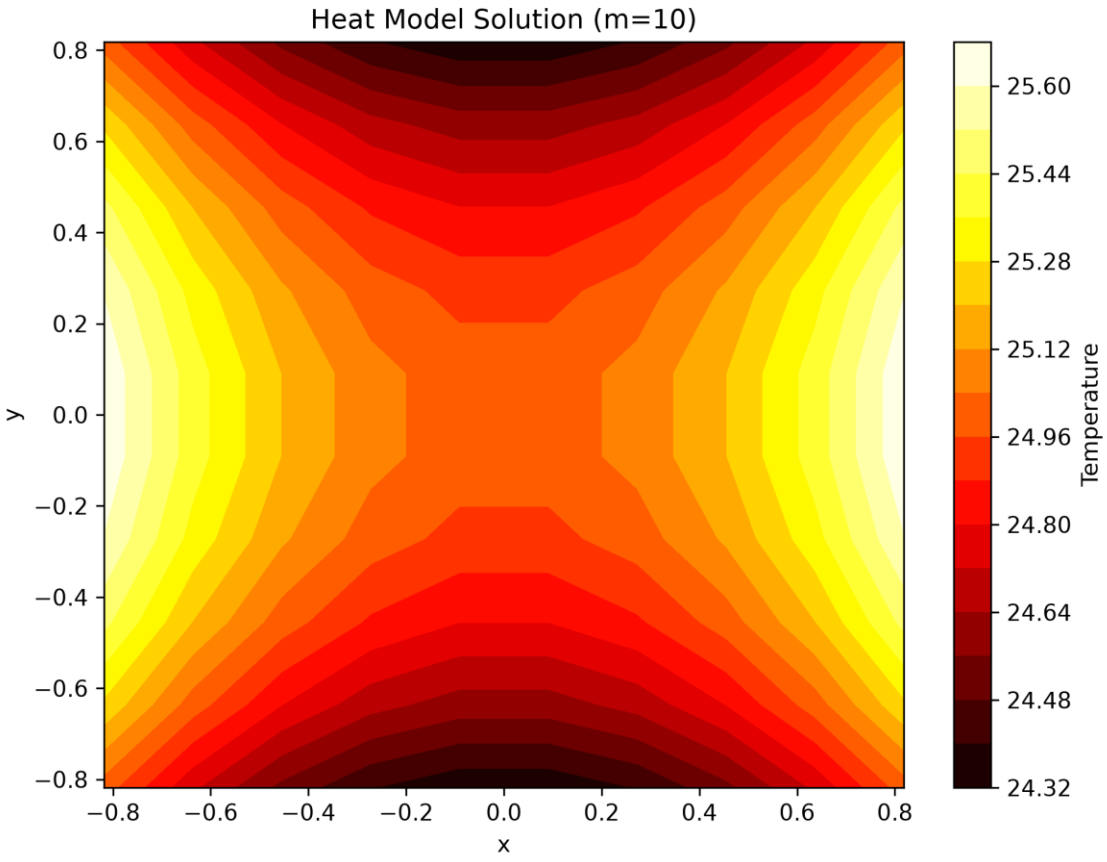


Figure 2.) Heatmap for m = 10

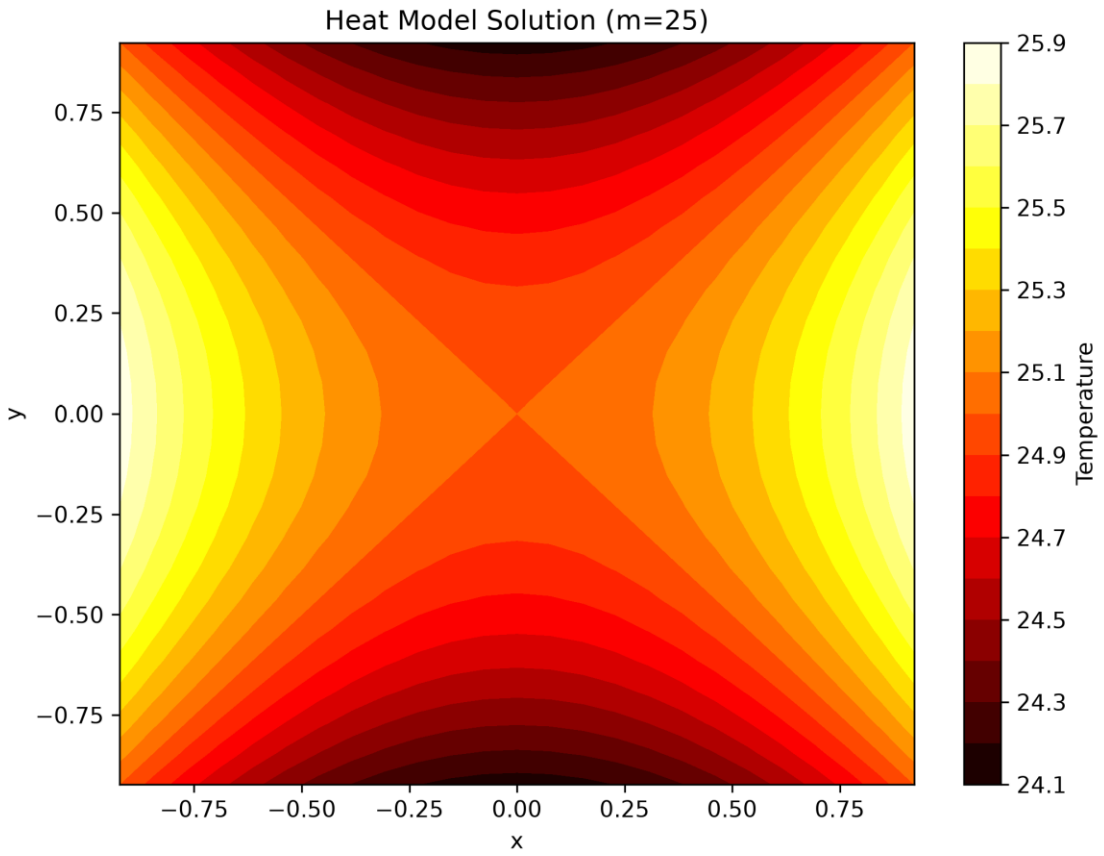


Figure 3.) Heatmap for m = 25

***Be advised, my code for the next part runs extremely slow due to the large amount of points and computations, but just let it do its thing!

Code:

```
import numpy as np

import matplotlib.pyplot as plt


def tridiag(m): #Creates m+m tridiagonal matrix, main diag values are all -4, above & below are 1

    T = np.diag(-4 * np.ones(m)) + np.diag(np.ones(m-1), 1) + np.diag(np.ones(m-1), -1)

    return T
```

```
def A_block(m): #Constructs the m^2 by m^2 block matrix A with T on main diag and Identity above and below
    T = tridiag(m) #Finds tridiag T of matrix size m
    I = np.identity(m) #Creates mxm identity matrix

    A = np.zeros((m*m, m*m)) #Constructs m^2 by m^2 matrix of zeros

    for i in range(m): #Fill in the A structure
        row_start = i * m
        row_end = (i+1) * m
        A[row_start:row_end, row_start:row_end] = T #Place T on main diag

        if i < m - 1:
            col_start = (i+1) * m
            col_end = (i + 2) * m
            A[row_start:row_end, col_start:col_end] = I #Place identity on surrounding diagonals

        if i > 0:
            col_start = (i - 1) * m
            col_end = i * m
            A[row_start:row_end, col_start:col_end] = I
    return A

def vector_b(m): #Constructs b vector using boundary conditions of f(x,y) given in homework
    b = np.zeros(m*m) #Fill it with zeros
    h = 2/ (m+1) #Calculate mesh spacing givem m points, two boundary points (-1 and 1) and total length of 2

    for i in range(m): #Loop over x
```

```
        for j in range(m): #Loop over y
            x = -1 + (i + 1) * h #Calculate x and y coordinates ofcurrent grid point
            y = -1 + (j + 1 )* h

            if j == m - 1: #Calculate boundary values at each (-1, 1) for both x and y
                b[i*m + j] -= (25 - x**2 + 1**2)

            if j == 0:
                b[i*m + j] -= (25 - x**2 + (-1)**2)

            if i == m - 1:
                b[i*m + j] -= (25 - 1**2 + y **2)

            if i == 0:
                b[i*m + j] -= (25 - (-1)**2 + y**2)
    return b

def cholesky(A): #Computes cholesky factorization, L is lower triangular
    n = A.shape[0]
    L = np.zeros_like(A)

    for i in range(n): #Loop over row
        for j in range(i+1): #Loop over column up diag
            s = sum(L[i][k] * L[j][k] for k in range(j)) #Calculate sum of products of prev columns

            if i == j:
                L[i][j] = np.sqrt(A[i][i] - s) #if on the diag, take square root
            else:
                L[i][j] = (A[i][j] - s) / L[j][j] #If not, calculate off-diag element
```

```
    return L


def solve_cholesky(L, b): #Solve Ax=6 using cholesky factor L via 1.) - Forward substitution and 2.) Back substitution


    n = len(b) #Forward sub, solve Lz=b for z

    z = np.zeros(n)

    for i  in range(n):

        z[i] = (b[i] - sum(L[i][j] * z[j] for j in range(i))) / L[i][i] #calculate z[i] using previously computed values


    x = np.zeros(n) #Back sub, solve L^T *x = z for x

    for i in range( n - 1, -1, -1):

        x[i] = (z[i] - sum(L[j][i] * x[j] for j in range(i + 1, n))) / L[i][i] #Calculate x[i] using previously computed values

        #L^T[i][j] = L[j][i], so sum is over j>i

    return x


def solve_plot(m): #Solves heat model problem and plot solution


    A = A_block(m) #Construct Matrix A and vector B

    b =vector_b(m)


    A = -A #Negate A and b to make A positive definite for Cholesky

    b = -b


    L = cholesky(A) #Compute cholesky factorization

    u_flat = solve_cholesky(L, b) #Solve system using cholesky

    u_grid = u_flat.reshape(m, m) #Reshape solution vector into 2D grid
```

```
    h = 2/(m+1) #Create coords for plots
    x = np.linspace(-1 + h, 1 - h, m)
    y = np.linspace(-1 + h, 1 - h, m)
    X, Y = np.meshgrid(x,y)

    plt.figure(figsize=(8,6)) #Create plot
    plt.contourf(X, Y, u_grid, levels=20, cmap = 'hot') #Creates filled contour plot
    plt.colorbar(label = 'Temperature') #Adds color bar for temp scale
    plt.title(f'Heat Model Solution (m={m})') #Adds title and labels
    plt.xlabel('x')
    plt.ylabel('y')
    plt.show() #display

solve_plot(10) #Solve and plot for m = 10 and m = 25
solve_plot(25)
```